BÀI 4: HÀNG ĐỢI



Mục tiêu
Sau khi học bài này, các bạn có thể: • Mô tả khái niệm về hàng đợi và phân biệt được hàng đợi với ngăn xếp. • Đặc tả về hàng đợi. • Trình bày chính xác các cách cài đặt hàng đợi bằng mảng và bằng danh sách liên kết được thực hiện bằng ngôn ngữ lập trình cụ thể. • Sử dụng cấu trúc dữ liệu dạng hàng đợi để giải các bài toán.

Nội dung
• Khái niệm về hàng đợi. • Đặc tả hàng đợi. • Các phương án cài đặt hàng đợi. • Hàng đợi vòng tròn. • Ứng dụng của hàng đợi.

Thời lượng học
8 tiết

Chúng ta hãy tưởng một hàng người đang chờ tính tiền ở một siêu thị, một dãy các máy bay đang chờ hạ cách xuống một sân bay, hay một dãy các công việc trong một hệ máy tính đang chờ một thiết bị ra nào đó, chẳng hạn như máy in. Trong các ví dụ trên, chúng ta nhận thấy chúng có một điểm chung đó là các mục trong mỗi hệ thống đó được phục vụ theo thứ tự như khi chúng đến; nghĩa là mục đầu tiên trong hàng sẽ được phục vụ đầu tiên.

4.1. Khái niệm về hàng đợi

Khái niệm

Kiểu dữ liệu hàng đợi là kiểu dữ liệu trong đó phần tử được đưa vào trước sẽ được loại bỏ trước. Kiểu dữ liệu hàng đợi còn được gọi là kiểu dữ liệu FIFO (First - In First - Out), có nghĩa là vào trước ra trước.

Hãy hình dung kiểu dữ liệu hàng đợi giống như một hàng đợi trong thực tế: Một hàng người xếp hàng đợi (hàng đơn) để chờ mua vé. Người nào xếp hàng trước sẽ được mua vé trước. Cũng giống như thế, phần tử nào được đưa vào hàng đợi trước sẽ được lấy ra trước để xử lý. Ngược lại, người nào đến sau để xếp hàng vào mua vé sẽ phải đứng vào cuối hàng, và sẽ phải chờ để mua vé sau những người đến trước. Tương tự, một phần tử mới được đưa vào hàng đợi sẽ được xếp vào cuối hàng đợi.

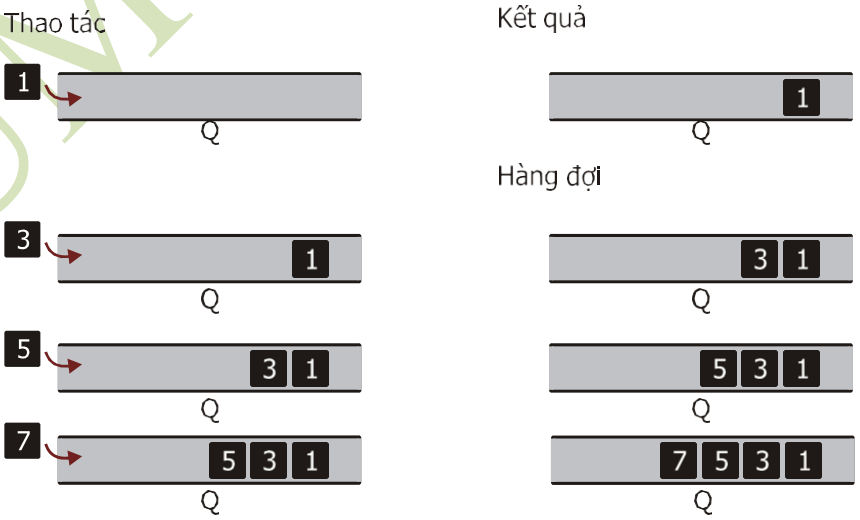
4.2. Đặc tả hàng đợi

Để hình dung rõ hơn một hàng đợi, ta hãy xem xét một ví dụ cụ thể như sau:

Giả sử ta có một hàng đợi Q ban đầu không có phần tử nào:



Tiếp theo ta thêm các phần tử vào hàng đợi. Phần tử nào vào trước sẽ được xếp bên phải, phần tử nào vào sau sẽ được xếp bên trái. Giả sử ta thêm lần lượt các phần tử có giá trị bằng 1, 3, 5, 7.



Như vậy sau 4 thao tác thêm phần tử, hàng đợi Q hiện thời có chứa 4 phần tử 1, 3, 5, 7. Giả sử ta cần lấy một phần tử từ hàng đợi ra để xử lý. Theo quy tắc xây dựng hàng

AUM VIỆT NAM

đợi, phần tử được lấy ra là phần tử được xếp vào đầu tiên. Vì vậy trong trường hợp này, phần tử được lấy ra chính là phần tử 1. Khi đó, hàng đợi sẽ còn lại 3 phần tử: 3, 5, 7.



Thao tác lấy phần tử tiếp theo sẽ lấy ra phần tử đầu tiên, khi này là phần tử 3. Như vậy hàng đợi chỉ còn 2 phần tử là 5 và 7.



Giả sử ta muốn thêm một phần tử vào hàng đợi, phần tử đó sẽ được xếp vào cuối hàng đợi, hay xếp ngay sau phần tử 7.

Cũng giống như kiểu dữ liệu ngăn xếp, hai thao tác cơ bản đối với kiểu dữ liệu là đưa một phần tử vào và lấy một phần tử ra khỏi cấu trúc dữ liệu. Điểm khác biệt giữa hai kiểu dữ liệu này chính là thứ tự của các phần tử khi vào và ra khỏi cấu trúc. Đối với kiểu dữ liệu hàng đợi, thao tác xếp vào hàng đợi (**enqueue**) sẽ đưa một phần tử vào cuối danh sách. Ngược lại, thao tác lấy một phần tử ra khỏi hàng đợi (**dequeue**) sẽ loại bỏ phần tử ở đầu danh sách.

4.3. Các phương án cài đặt hàng đợi

Trên thực tế có nhiều phướng án cài đặt hàng đợi để đạt được tính hiệu quả về bộ nhớ và tốc độ xử lý. Trong khuôn khổ giáo trình này, chúng tôi chỉ giới thiệu hai phương pháp cài đặt sử dụng kiểu dữ liệu mảng và kiểu dữ liệu danh sách liên kết. Mỗi một phương án cài đặt đều mang những ưu điểm và nhược điểm riêng của mình.

4.3.1. Cài đặt hàng đợi bằng mảng

Cài đặt hàng đợi bằng mảng là phương pháp cài đặt hàng đợi đơn giản nhất. Các phần tử của hàng đợi được lưu trữ lần lượt bằng các phần tử của mảng.

Lưu ý đối với phương pháp này, mảng là kiểu dữ liệu tĩnh, kích thước của mảng là cố định. Vì vậy ta không thể lưu trữ nhiều phần tử hơn kích thước của mảng. Mỗi lần thêm một phần tử vào hàng đợi, phần tử đó sẽ được thêm vào cuối danh sách phần tử. Vì vậy cần phải có một biến để ghi nhớ vị trí của phần tử cuối cùng trong hàng đợi. Ở đây ta sử dụng một biến có kiểu NGUYÊN với tên là T (viết tắt của thuật ngữ tiếng Anh Tail có nghĩa là đuôi) để chỉ vị trí phần tử tiếp theo sẽ được thêm vào hàng đợi. Mỗi lần thêm một phần tử mới vào hàng đợi, nó sẽ được xếp vào vị trí T và giá trị của Tsẽ được tăng lên 1.

Khác với kiểu dữ liệu ngăn xếp, phần tử được lấy ra từ cuối dãy, hay chính là phần tử ở vị trí T, vì thế vị trí của phần tử đầu tiên của ngăn xếp luôn là phần tử đầu tiên của mảng. Đối với hàng đợi, phần tử được lấy ra ở đầu dãy. Vì vậy phần tử đứng đầu

AUM VIỆT NAM

hàng đợi không phải lúc nào cũng là phần tử đầu tiên của mảng. Để đảm bảo phần tử đứng đầu hàng đợi luôn là phần tử đầu tiên của mảng ta có thể thực hiện di chuyển các phần tử của ngăn xếp lên đầu sau mỗi thao tác lấy phần tử ra khỏi hàng đợi. Tuy nhiên việc thay đổi vị trí các phần tử trong mảng tốn khá nhiều thời gian chạy chương trình, vì vậy sẽ làm cho chương trình kém hiệu quả. Thay vào đó chúng ta sẽ sử dụng một biến khác để lưu trữ vị trí của phần tử đứng đầu hàng đợi Biến này cũng có kiểu NGUYÊN và có tên là H(viết tắt của thuật ngữ tiếng Anh Head có nghĩa là đầu). Mỗi lần loại bỏ một phần tử ra khỏi hàng đợi, giá trị của Hsẽ được tăng lên 1.

Giá trị ban đầu của H bằng 1 để cho thấy rằng ban đầu hàng đợi được lưu trữ bắt đầu từ phần tử đầu tiên của mảng. Giá trị ban đầu của T được gán bằng 1 để cho thấy phần tử đầu tiên xếp vào hàng đợi sẽ được xếp vào vị trí số 1.

Ngoài ra, với mục đích kiểm soát khả năng lưu trữ của hàng đợi, chúng ta sử dụng thêm một biến có tên là S (viết tắt của thuật ngữ size trong tiếng Anh) để lưu số phần tử hiện thời có trong hàng đợi. Vì vậy hàng đợi rỗng tương đương với S = 0. Hay giá trị ban đầu của S = 0vì ban đầu hàng đợi là rỗng.

Chúng ta sử dụng mảng để cài đặt hàng đợi, trong khi số lượng phần tử trong một mảng là cố định, ở đây chúng ta sử dụng biến có tên là kich_thuoc (capacity) để lưu kích thước của mảng, nên số phần tử của hàng đợi không được phép vượt quá số phần tử của mảng.

Khi số phần tử của hàng đợi bằng số phần tử của mảng, S = kich_thuoc, hàng đợi được gọi là đầy (full), và không thể thêm được phần tử nào nữa vào hàng đợi.

Giả sử các phần tử của ngăn xếp có kiểu là ELEMENT (ELEMENT có thể là NGUYÊN, KÝ TỰ, THỰC, PHỨC, ...), kiểu dữ liệu hàng đợi với dung lượng lưu trữ tối đa là 10 phần tử và được cài đặt bằng mảng có thể được định nghĩa như sau

```
typedef int ELEMENT;
struct QUEUE_ARRAY
        {
         ELEMENT ele[10];
         int capacity, H, T, S ;
        };
```

Việc khởi tạo hàng đợi sẽ được viết như sau:

```
QUEUE_ARRAY INIT_QUEUE_ARRAY(Struct QUEUE_ARRAY q)
  {
    q.capacity = 10;
    q.H = -1;
    q.T = -1;
    q.S = 0;
    return q;
  }
```

Ngoài ra có hai giải thuật phụ để kiểm tra trạng thái của hàng đợi (rỗng hoặc đầy).

AUM VIỆT NAM

```
int IS_EMPTY(QUEUE_ARRAY q)
/*q đã được khởi tạo bằng hàm
INIT_QUEUE_ARRAY(QUEUE_ARRAY  q)
*/
{
   int empty;
   empty = 0;
   if (q.S == 0)
                empty = 1;
   else
                empty = 0;
   return
                empty;
}
```

```
int IS_FULL(QUEUE_ARRAY q)
/*q đã được khởi tạo bằng hàm
INIT_QUEUE_ARRAY(QUEUE_ARRAY
q)*/
{
   int full;
   full = 0;
   if (q.S == q.capacity)
                full = 1;
   else
                full = 0;
   return
                full;
}
```

Trước khi cài đặt các thao tác trên kiểu dữ liệu HANG_DOI_MANG ta hãy xem xét một ví dụ cụ thể trên hàng đợi.

Giả sử ta có một hàng đợi q thuộc kiểu HANG_DOI_MANG với dung lượng lưu trữ tối đa kich_thuoc = 4.

Sau khi gọi thuật toán khởi tạo, hàng đợi rỗng (không có phần tử nào), và H = 1, T = 1, S = 0. Lần lượt thêm các phần tử có giá trị bằng 1, 2, 3 vào hàng đợi (xem bảng dưới). Ở đây ta sử dụng ký tự X để biểu thị một phần tử với giá trị bất kỳ. Mảng elements của q sẽ có 4 phần tử.

Thao tác	q.elements	Trạng thái
Khởi tạo	X X X X	H = 1 T = 1 S = 0
Thêm phần tử 1	1 X X X	H = 1 T = 2 S = 1
Thêm phần tử 2	1 2 X X	H = 1 T = 3 S = 2
Thêm phần tử 3	1 2 3 X	H = 1 T = 4 S = 3

Ta có thể thấy rằng mỗi lần thêm 1 phần tử mới vào hàng đợi, giá trị của H không thay đổi hay vị trí của phần tử đầu tiên của hàng đợi ở trong mảng là không thay đổi. Trong khi đó giá trị của Ssẽ tăng lên một vì số phần tử của hàng đợi sẽ tăng lên 1. Giá trị của T cũng sẽ tăng lên 1, vì khi một phần tử được thêm vào hàng đợi, nó sẽ được lưu tại vị trí T của mảng, do đó phần tử tiếp theo được thêm vào hàng đợi sẽ phải được lưu vào vị trí T + 1. Ngược lại ta hãy xem thao tác lấy một phần tử ra khỏi hàng đợi.

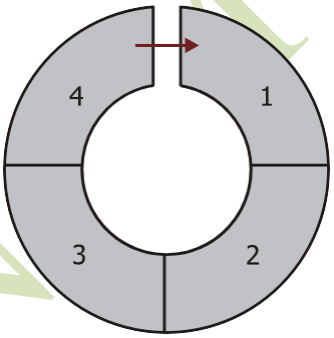
Thao tác	q.elements	Trạng thái
Ban đầu	1 2 3 X	H = 1 T = 4 S = 3
Lấy 1 phần tử ra khỏi hàng đợi	X 2 3 X —> phần tử lấy ra: 1	H = 2 T = 4 S = 2

Phần tử được lấy ra sẽ là phần tử nằm ở vị trí H của mảng. Vì vậy, sau khi phần tử này được lấy ra, phần tử đầu tiên của hàng đợi phải là phần tử nằm ở vị trí H + 1của

AUM VIỆT NAM

mảng, do đó giá trị của H được tăng lên 1. Hiển nhiên giá trị của S sẽ bị giảm đi 1 vì 1 phần tử đã bị lấy ra khỏi hàng đợi. Giá trị của Tkhông thay đổi.

Giả sử ta muốn thêm 2 phần tử 4 và 5 vào hàng đợi. Tương tự như trên, phần tử 4 sẽ được thêm vào vị trí T = 4 của mảng. Khi đó H = 2, S = 3, T = 5. Do đó phần tử tiếp theo được thêm vào hàng đợi sẽ được thêm vào vị trí thứ 5 của mảng. Tuy nhiên kích thước của mảng bằng 4 nên vị trí thứ 5 là không hợp lệ, hay nói cách khác ta không thể thêm phần tử nữa vào mảng. Điều này là bất hợp lý vì số phần tử của hàng đợi S = 3 vẫn nhỏ hơn kích thước của mảng, và trong mảng vẫn còn 1 phần tử trống chưa sử dụng ở vị trí số 1.



Để giải quyết vấn đề này, ta hãy hình dung mảng giống như 1 vòng tròn, các phần tử sẽ được xếp thứ tự theo chiều kim đồng hồ. Phần tử cuối cùng của mảng sẽ được nối với phần tử đầu tiên của mảng. Khi đó phần tử đầu tiên của mảng sẽ được coi như phần tử đứng tiếp sau phần tử cuối cùng của mảng. Khi đó, trong trường hợp T = kich_thuoc, nếu ta thêm một phần tử vào hàng đợi, phần tử này sẽ được xếp vào vị trí cuối cùng của mảng.

Giá trị của T sẽ được gán bằng 1 để cho biết rằng phần tử tiếp theo được thêm vào hàng đợi sẽ được xếp vào vị trí đầu tiên của mảng.

Với cách sắp xếp này, sẽ có trường hợp phần tử đứng đầu hàng đợi được xếp ở cuối mảng hay H = kich_thuoc, còn các phần tử đứng sau lại được xếp trên đầu mảng. Khi đó, nếu thực hiện thao tác lấy một phần tử ra khỏi hàng đợi, phần tử được lấy ra sẽ là phần tử đứng cuối mảng. Phần tử đứng ngay sau nó, bây giờ là phần tử đứng đầu mảng sẽ trở thành phần tử đầu tiên của hàng đợi. Vì vậy Hsẽ phải được gán giá trị mới bằng 1.

Với các hiện tượng quan sát ở trên, các giải thuật thêm phần tử vào hàng đợi và rút phần tử ra khỏi hàng đợi có thể được viết như sau:

```
void ENQUEUE(QUEUE_ARRAY q,ELEMENT e)
{
 if (IS_FULL(q)!= 0)
                printf("hang doi day khong the chen them");
 else
        {
                 if (q.T == q.capacity - 1)
                                q.T = 0;
                 else
                                q.T = q.T + 1;
                                q.ele[q.T] = e;
                                q.S = q.S + 1;
        }
}
```

AUM VIET NAM

```
ELEMENT DEQUEUE(QUEUE_ARRAY q)
{
 ELEMENT e;
 if (IS_EMPTY(q)!= 0)
        printf("hang doi rong khong the lay phan tu ra");
 else
        {
                e = q.ele[q.H];
                if (q.H == q.T)
                                {
                                 q.H = -1;
                                 q.T = -1;
                                }
                else
                                 q.H = q.H + 1;
                                 q.S = q.S - 1;
        }
 return e;
}
```

Đặc tính của hàng đợi cài đặt bằng mảng:

Cũng như đối với ngăn xếp, việc sử dụng mảng để cài đặt hàng đợi cho phép tận dụng các ưu điểm của mảng đó là thời gian truy xuất nhanh vì các phần tử của mảng được truy xuất một cách trực tiếp. Vì vậy các thao tác trên hàng đợi cài đặt bằng mảng có thời gian thực hiện nhanh.

Tuy nhiên, hạn chế của việc sử dụng mảng để cài đặt hàng đợi là số phần tử của mảng là cố định, vì vậy kích thước của hàng đợi cũng bị giới hạn (không được vượt quá kích thước của mảng). Vì vậy phương pháp cài đặt này thường được sử dụng trong các trường hợp mà số phần tử tối đa của hàng đợi đã được biết trước và không quá lớn.

Trong phần sau của bài chúng ta sẽ xem xét một phương pháp cài đặt hàng đợi khác, cho phép cài đặt một hàng đợi với kích thước bất kỳ.

4.3.2. Cài đặt hàng đợi bằng danh sách liên kết

Để cài đặt hàng đợi bằng danh sách liên kết, trước tiên ta phải định nghĩa kiểu phần tử cho danh sách. Mỗi phần tử của danh sách liên kết phải có:

- 1 trường để lưu thông tin về phần tử trong hàng đợi (kiểu ELEMENT).
- 1 con trỏ để chỉ tới phần tử tiếp theo trong danh sách.

Do đó ta có thể định nghĩa kiểu cho các phần tử trong danh sách như sau:

```
typedef int ELEMENT;
struct LIST_ELEMENT
 {
          ELEMENT info;
          struct LIST_ELEMENT *next;
 };
```

AUM VIỆT NAM

Để sử dụng danh sách liên kết chỉ cần lưu phần tử đầu tiên của danh sách. Vì vậy trong kiểu cấu trúc hàng đợi mới chúng ta sẽ sử dụng một con trỏ Htrỏ tới phần tử đầu tiên của danh sách. Nếu danh sách rỗng, không có phần tử nào trong hàng đợi, con trỏ đó sẽ có giá trị RỖNG (NULL). Một biến S kiểu NGUYÊN cũng được sử dụng để lưu số phần tử hiện thời trong hàng đợi.

```
struct QUEUE_LIST
 {
        struct LIST_ELEMENT *H;
        int S;
 };
```

Việc khởi tạo một hàng đợi sử dụng danh sách liên kết cũng chỉ đơn giản là gán trường H của nó bằng RỖNG (danh sách rỗng), và trường S bằng 0 vì chưa có phần tử nào.

```
QUEUE_LIST INITIALIZE(QUEUE_LIST q)
 {
    q.H = NULL;
    q.S = 0;
    return q;
 }
```

Vì kiểu dữ liệu danh sách cho phép lưu trữ một số lượng phần tử bất kỳ (với điều kiện bộ nhớ cho phép), nên không xảy ra trường hợp hàng đợi bị đầy.

Trạng thái hàng đợi rỗng có thể được kiểm tra bằng cách kiểm tra số phần tử của hàng đợi được lưu trong biến S.

Thao tác xếp vào hàng đợi (enqueue)

Một phần tử mới được xếp vào hàng đợi sẽ nằm ở cuối hàng đợi, hay nằm ở cuối danh sách q.noi_dung. Vì vậy việc đưa một phần tử vào hàng đợi chính là tạo và đưa một phần tử mới vào danh sách.

Nếu danh sách hiện thời đang rỗng, H = NULL, ta chỉ việc tạo một phần tử mới cho danh sách và gán giá trị cho Hbằng địa chỉ của phần tử mới này.

Nếu danh sách không rỗng, H <> NULL, việc đầu tiên là phải xác định phần tử nằm cuối danh sách hiện thời và xếp phần tử mới vào sau phần tử đó.

Trong cả hai trường hợp ta đều phải thực hiện việc tạo một phần tử cho danh sách và lấy địa chỉ của phần tử đó. Nhắc lại, thao tác lấy địa chỉ của một biến chính là: ADDR (viết tắt của Address). Để tạo một phần tử mới ta sử dụng câu lệnh TẠO MỚI (hay NEW trong tiếng Anh). Giá trị trả về của thao tác này chính là địa của vùng bộ nhớ lưu phần tử mới được tạo ra. Do đó, trên thực tế ta không cần phải sử dụng câu lệnh lấy địa chỉ.

Khác với kiểu cấu trúc mảng, các phần tử trong danh sách không được truy cập một cách trực tiếp mà phải truy cập lần lượt từng phần tử một. Vì vậy để xác định phần tử đứng cuối danh sách ta phải duyệt qua toàn bộ danh sách. Vì số phần tử trong danh sách đã biết trước, chính là giá trị của trường S, nên ta có thể sử dụng vòng lặp CHO để duyệt qua toàn bộ danh sách.

AUM VIỆT NAM

Dưới đây là mô tả chi tiết giải thuật thêm phần tử vào hàng đợi:

```
QUEUE_LIST ENQUEUE(QUEUE_LIST q,ELEMENT e)
{
  LIST_ELEMENT  *p,*m;
  m = new LIST_ELEMENT;
  m -> info = e;
  m -> next = NULL;
  if (q.S == 0)
                q.H = m;
  else
          {
                 p = q.H;
                 for (int i = 0; i < q.S - 2; i++)
                         p = p -> next;
                 p -> next = m;
          }
  return q;
}
```

Ban đầu một phần tử kiểu LIST_ELEMENTsẽ được khởi tạo, hay cấp phát bộ nhớ. Phần tử này sẽ mang giá trị bằng elà giá trị cần thêm vào hàng đợi.

Câu lệnh so sánh q.S == 0 tương đương với việc kiểm tra hàng đợi rỗng (số phần tử bằng 0). Nếu hàng đợi rỗng thì phần tử đầu tiên của hàng đợi chính là phần tử mới được tạo ra. Vì vậy giá trị của q.H chính bằng m.

Ngược lại, nếu hàng đợi không rỗng ta sử dụng thêm một biến phụ p kiểu CON TRỎtới LIST_ELEMENT. Ban đầu p được gán bằng q.H do đó p trỏ đến phần tử đầu tiên của danh sách. Mỗi lần thực hiện câu lệnh gán p <— p –> next con trỏ p sẽ chỉ đến phần tử tiếp theo. Vì vậy sau q.S – 1 lần dịch chuyển, q.S chính là số phần tử của danh sách, thì p sẽ trỏ đến phần tử cuối cùng của danh sách. Cuối cùng ta chỉ việc đặt mlà phần tử tiếp theo của p. Khi đó mtrở thành phần tử nằm cuối danh sách.

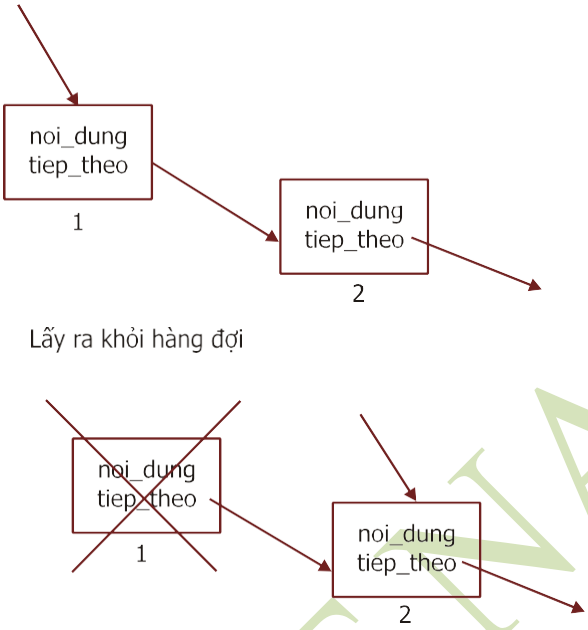
Mỗi lần thêm 1 phần tử vào hàng đợi ta phải duyệt toàn bộ danh sách do đó làm tăng thời gian thực hiện thao tác lên rất nhiều. Để giải quyết vấn đề này ta có thể thêm một trường nữa để lưu vị trí cuối danh sách T. Ban đầu Tbằng rỗng do chưa có phần tử nào trong danh sách. Khi thêm phần tử đầu tiên vào danh sách, cả H và T đều trỏ vào phần tử đó (phần tử đầu cũng chính là phần tử cuối). Nhờ có trường T này, mỗi lần thêm một phần tử vào hàng đợi không rỗng ta chỉ việc đưa phần tử đó vào sau T (do lúc này T là phần tử đứng cuối cùng) và cập nhật lại giá trị của Tđể trỏ tới phần tử đứng cuối mới.

Thao tác lấy ra khỏi hàng đợi (dequeue)

Phần tử được lấy ra là phần tử đầu tiên của hàng đợi, hay chính là phần tử được trỏ bởi con trỏ H. Như vậy phần tử đầu tiên của hàng đợi bây giờ chính là phần tử đứng

ÁI VIỆT NAM

ngay sau phần tử được trỏ bởi con trỏ H, hay chính là phần tử được quy định bởi trường next. Do đó, giá trị mới của H phải được gán bằng H –> next (xem hình minh họa).



Vì vậy thao tác lấy phần tử ra khỏi hàng đợi sẽ được cài đặt như sau:

```
QUEUE_LIST DEQUEUE(QUEUE_LIST q)
 {
  ELEMENT e;
  if (q.S == 0)
    printf("hang doi rong khong the lay phan tu");
  else
          {
                  e = q.H -> info;
                  q.S = q.S - 1;
                  q.H = q.H -> next;
          }
  return e;
 }
```

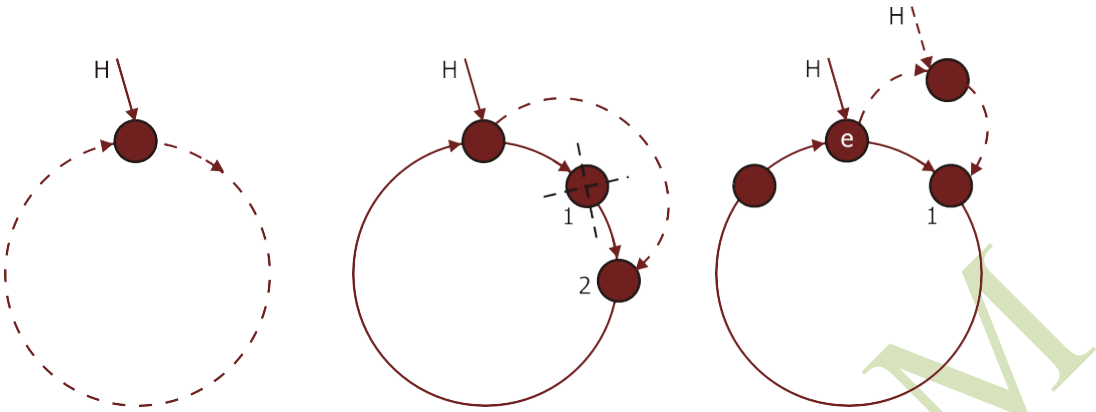
Thuật toán trên có sử dụng một câu lệnh tương đối phức tạp là q.H –> noi_dung. Câu lệnh trên được giải thích như sau:

q.Hđể tham chiếu tới con trỏ Hcủa hàng đợi q

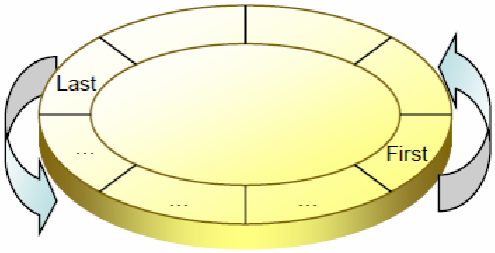
- q.H thuộc kiểu CON TRỎ tới LIST_ELEMENT, vì vậy để truy xuất tới các trường của bản ghi được trỏ bởi q.Hchúng ta sử dụng phép toán tham chiếu –>
- q.H –> info: để lấy về giá trị của trường info trong bản ghi được trỏ bởi q.H
- q.H –> next: để lấy về giá trị của trường next trong bản ghi được trỏ bởi q.H

AUM VIỆT NAM

4.4. Hàng đợi vòng tròn



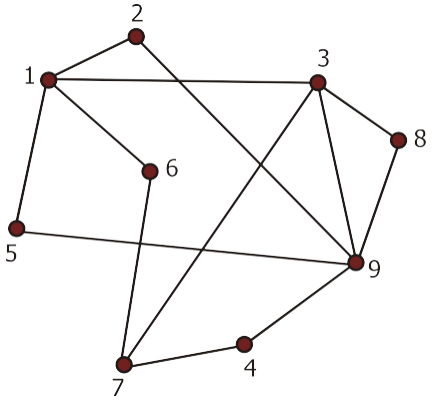
Trong cách cài đặt hàng đợi như trên ta nhận thấy khi thực hiện thao tác xếp một phần tử mới vào hàng thì đến một lúc nào đó ta sẽ nhận được thông báo tràn mảng (hoặc tràn bộ nhớ khi cài đặt bằng danh sách liên kết dạng con trỏ) do trong thao tác này mỗi lần xếp một phần tử mới vào hàng thì biến T (chỉ tới phần tử cuối hàng đợi) cũng tăng và không bao giờ bị giảm đi cả. Để khắc phục nhược điểm này bằng một danh sách vòng (biểu diễn bằng mảng hoặc danh sách liên kết), coi như các phần tử của hàng đợi được xếp quanh vòng theo một hướng nào đó. Các phần tử nằm trên phần cung tròn từ vị trí H tới vị trí T là các phần tử của hàng đợi. Có thêm một biến S lưu số phần tử trong hàng đợi. Khi đó hàng đợi rỗng hoặc đầy khi H bằng T. Việc thêm một phần tử vào hàng đợi tương đương với việc ta dịch chỉ số T theo vòng một vị trí rồi đặt giá trị mới vào đó. Việc loại bỏ một phần tử trong hàng đợi tương đương với việc lấy ra một phần tử tại vị trí H rồi dịch chỉ số H theo vòng.



4.5. Ứng dụng của hàng đợi

Ngăn xếp và hàng đợi là hai kiểu cấu trúc dữ liệu được dùng nhiều trong thực tế. Một trong những ứng dụng thường gặp nhất là trong lý thuyết đồ thị. Ngăn xếp và hàng đợi được sử dụng để duyệt đồ thị theo chiều sâu và chiều rộng.

Ví dụ, có một đồ thị được cho trong hình dưới. Vấn đề đặt ra là tìm đường đi ngắn nhất từ đỉnh 2 đến đỉnh 7.



AUM VIỆT NAM

Bằng việc sử dụng hàng đợi, ta có thể tìm được đường đi ngắn nhất từ 2 đến 7 bằng thuật toán duyệt đồ thị theo chiều rộng như sau:

- **Bước 1:** Khởi tạo một hàng đợi Q rỗng
- **Bước 2:** xếp phần tử 2 vào hàng đợi Q.
- **Bước 3:** lần lượt lấy các phần tử ra khỏi hàng đợi Q và xếp tất cả các đỉnh kề với nó (trừ những đỉnh đã xét) vào hàng đợi cho đến khi gặp được phần tử có giá trị bằng 7.

Nếu không còn phần tử nào trong hàng đợi mà chưa tìm được phần tử có giá trị bằng 7 thì kết luận không có đường đi từ 2 đến 4.

Kết quả của thuật toán trên được mô tả trong bảng sau:

Thao tác	Kết quả Q
Xếp 2 vào hàng đợi	2
Lấy 1 phần tử ra khỏi hàng đợi: 2	rỗng
Xếp tất cả các phần tử kề với 2 vào Q	1, 9
Lấy 1 phần tử ra khỏi hàng đợi: 1	9
Xếp tất cả các phần tử kề với 1 vào Q	9 3 5 6
Lấy 1 phần tử ra khỏi hàng đợi: 9	3 5 6
Xếp tất cả các phần tử kề với 9 vào Q	3 5 6 4 8 (2 và 5 đã được xét nên không được xếp vào)
Lấy 1 phần tử ra khỏi hàng đợi: 3	5 6 4 8
Xếp tất cả các phần tử kề với 3 vào Q	5 6 4 8 7

Như vậy sau 9 thao tác ta đã gặp được phần tử có giá trị bằng 7. Lần ngược lại các thao tác ta sẽ thu được 1 đường đi ngắn nhất từ 2 đến 7:

- 7 được xếp vào Q do liền kề với 3
- 3 được xếp vào Q do liền kề với 1
- 1 được xếp vào Q do liền kề với 2

Vậy đường đi ngắn nhất từ 2 đến 7 tìm được là: 2 1 3 7.

TÓM LƯỢC CUỐI BÀI

Trong bài này chúng ta tìm hiểu một dạng cấu trúc dữ liệu đặc biệt của danh sách. Hàng đợi còn gọi là danh sách hạn chế. Nên các bạn cần lưu ý một số vấn đề sau:

- Trình bày đúng khái niệm về hàng đợi. Phân biệt điểm giống và khác giữa hàng đợi và danh sách.
- Mô tả đúng các đặc tả về hàng đợi và các phương án cài đặt cho hàng đợi.
- Trình bày được các ứng dụng của hang đợi và vận dụng hàng đợi vào giải quyết các bài toán.

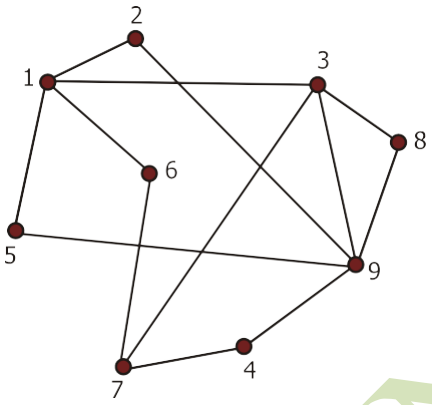
AUM VIỆT NAM

BÀI TẬP

1. Phân biệt ngăn xếp và hàng đợi?

2. Có thể sử dụng ngăn xếp để tìm đường đi ngắn nhất hay không?

3. Cho đồ thị dưới đây, hãy tìm đường đi ngắn nhất từ 1 đến 9 bằng cách sử dụng hàng đợi.



4. Sử dụng cấu trúc dữ liệu dạng hàng đợi. Viết chương trình chuyển một số từ hệ thập phân sang hệ cơ số 2.

AUM VIỆT NAM